

Технические требования: анимация маскота

Вертушки для приложения

Документ для дизайнера/аниматора и разработчика. Написан так, чтобы **не-технический человек** **понял**, какой формат делать и почему.

Контекст: маскот (пластинка-персонаж) собирается в Adobe Illustrator послойно → анимируется → кладётся в мобильное приложение Вертушка (Expo / React Native).

Куда пойдёт: **интро-экран** (заставка при первом запуске) и **экраны загрузки** (лоадеры внутри приложения).

0. Самое главное за 30 секунд

- Формат для приложения — Lottie (`.json`).** Не видео, не GIF, не PNG-последовательность. Почему — ниже.
- Аниматор делает анимацию в **After Effects**, экспортирует плагином **Bodymovin** в `.json`.
- Есть жёсткие **запреты** (что нельзя использовать в After Effects, иначе в приложении сломается) — раздел 4.
- В приложении сейчас **нет проигрывателя Lottie** — это отдельная задача разработчику (раздел 7). Без неё `.json` не заработает.
- Заставку (native splash) анимировать нельзя** — там только статичная картинка. Анимация начинается уже внутри приложения. Важное ограничение, объяснено в разделе 6.

1. Почему Lottie, а не видео/GIF

Приложение написано на **React Native (Expo)**. Для него:

Формат	Вес	Качество на любом экране	Прозрачный фон	Вердикт
Lottie (<code>.json</code>)	10–150 КБ	Идеально (вектор, не пикселизуется)	Да, из коробки	✅ Основной выбор
GIF	0.5–5 МБ	Плохо (256 цветов, рвётся градиент)	Кривая, с ореолом	❌ Нет
MP4-видео	1–10 МБ	Ок, но фиксированный размер	Почти нереально сделать	⚠️ Только интро без прозрачности
PNG-последовательность	2–20 МБ	Ок	Да	⚠️ Тяжело, крайний случай

Lottie — это анимация, записанная как **набор инструкций** («круг сдвинь сюда, поверни на 20°»), а не как видео из пикселей. Поэтому файл крошечный, а картинка чёткая на любом телефоне. Градиент маскота

(синий→розовый) останется гладким.

Вывод: делаем в Lottie. Видео держим в уме только как запасной вариант для интро (см. раздел 5).

2. Пайплайн: от Illustrator до приложения

```
Adobe Illustrator      After Effects      Приложение
(послойный маскот)    → (анимация слоёв)  → (.json Lottie)
                        ↓ плагин Bodymovin
                        экспорт в mascot.json
```

Пошагово для аниматора:

1. В Illustrator маскот уже собран послойно (голова-пластинка, руки-шары, ноги, кроссы). Каждая часть — на своём слое.
2. Импортировать в After Effects (**File** → **Import**, сохранить слои — «Composition, Retain Layer Sizes»).
3. Анимировать движением слоёв (позиция, поворот, масштаб). **Всё, что двигается, — это векторные фигуры (shape layers), а не картинки.**
4. Поставить плагин **Bodymovin** (он же LottieFiles for After Effects) — бесплатный, из Adobe Exchange или с lottiefiles.com.
5. Экспорт: окно Bodymovin → выбрать композицию → Render → получить `mascot.json`.
6. Проверить `.json` на lottiefiles.com/preview (перетащить файл в браузер) — **как выглядит там, так и будет в приложении**. Если там сломано — переделать, не отдавать разработчику.

3. Технические требования к файлам

Делаем **два разных файла** — у них разные задачи, разные бюджеты.

3.1 Интро-экран (`intro-mascot.json`)

Играет один раз при старте. Можно чуть тяжелее — пользователь видит редко.

Параметр	Значение
Формат	<code>.json</code> , экспорт Bodymovin
Версия Bodymovin	5.7 или новее
Частота кадров (FPS)	30 (не 60 — вдвое тяжелее, на телефоне незаметно)
Длительность	2–4 секунды
Проигрывание	Один раз, без лупа. В конце — статичная финальная поза
Артборд (Composition)	Квадрат 1080 × 1080 px
Центрирование	Маскот в центре, отступ от краёв $\geq 10\%$ (на узких телефонах не обрежется)
Вес файла	< 200 КБ (цель < 120 КБ)

3.2 Лоадер внутри приложения (`loader-mascot.json`)

Крутится, пока грузятся данные. Показывается **часто** → должен быть максимально лёгким и бесшовным.

Параметр	Значение
Формат	<code>.json</code> , экспорт Bodymovin
FPS	30
Длительность	1–2 секунды
Проигрывание	Бесшовный луп — первый кадр обязан точно совпадать с последним, иначе на стыке будет «рывок»
Артборд	Квадрат 512 × 512 px
Вес файла	< 100 КБ (цель < 60 КБ)

Про бесшовный луп простыми словами: если это «пластинка крутится» — за 1 секунду она должна повернуться ровно на 360° (или на угол, кратный симметрии рисунка), чтобы кадр замыкался сам на себя. Проверка: на превью луп не должен «дёргаться» на повторе.

4. 🚫 ЗАПРЕТЫ — что нельзя в After Effects

Lottie-проигрыватель в React Native **не умеет** часть эффектов After Effects. Если аниматор их использует — в приложении будет либо пусто, либо тормоза, либо артефакты. **Экспорт может пройти без ошибки, а сломается уже в приложении** — поэтому список важен.

Нельзя:

- ❌ **Растровые картинки внутри анимации** (вставленные `.png` / `.jpg`). Всё должно быть **векторными фигурами** (shape layers). Единственное исключение — раздел 4.1 про halftone.
- ❌ **Эффекты After Effects:** Blur (размытие), Glow (свечение), Drop Shadow, Gradient Ramp сложный, любые из вкладки Effects. Они **не экспортируются** в Lottie.

- **✗ Merge Paths** (объединение контуров) и **Auto-Trace**.
- **✗ Сложные маски и Track Mattes** (обтравка одного слоя другим) — минимизировать, лучше без них.
- **✗ Expressions** (скриптовые выражения на свойствах). Если использовали для удобства — перед экспортом «запечь» в **ключевые кадры** (Bodymovin умеет, но надёжнее вручную: правый клик → Convert Expression to Keyframes).
- **✗ Шрифты как текст**. Если в анимации есть надпись (например «Вертушка» или «Legendary» на кроссовке) — **перевести в кривые** (Create Outlines), не оставлять живой текст.
- **✗ 3D-слои и камеры** After Effects. Только 2D.

Можно (это и есть основа Lottie): позиция, поворот, масштаб, прозрачность (Opacity), обводка/заливка векторных фигур, Trim Paths (рисование контура), стандартные easing-кривые.

4.1 Отдельно про halftone (полутонные точки на теле маскота)

На рисунке тело маскота покрыто **точечным полутонном** (мелкие точки-градиент). Для векторного Lottie это **тяжело**: тысячи точек = огромный `.json` и тормоза.

Варианты (выбрать один, согласовать с разработчиком): 1. **Упростить** — заменить halftone на плоский градиент в анимируемых частях. Проще всего, легче всего. 2. **Запечь один раз** — halftone-слой отдать как один статичный оптимизированный элемент, который сам не анимируется (только двигается целиком). Компромисс. 3. Halftone только на **неподвижных** частях, а движущиеся (крутящаяся голова) — без него.

5. Запасной вариант: если анимацию делаете из 3D-рендера

Если маскот в итоге собирают в 3D (Blender/C4D) и оттуда рендерят — Lottie из этого **не получить**. Тогда:

5.1 Интро как видео (только если без прозрачного фона)

Параметр	Значение
Формат	<code>.mp4</code> , кодек H.264
Фон	Непрозрачный — залит фоновым цветом экрана (см. раздел 8). Прозрачное видео в React Native почти не работает, не закладываться на него
Разрешение	1080 × 1080 px
FPS	30
Вес	< 2 МБ
Проигрыватель	нужен <code>expo-video</code> (доп. задача разработчику)

5.2 Крутящаяся пластинка как PNG-последовательность (крайний случай)

Если нужен именно 3D-спин пластинки с прозрачным фоном:

Параметр	Значение
Формат	PNG-кадры с прозрачностью → собрать в один спрайт-атлас
Кадров на полный оборот	24–36
Размер кадра	512 × 512 px
Луп	бесшовный оборот 360°
Вес атласа	< 500 КБ (прогнать через TinyPNG/pngquant)

⚠ Оба варианта из раздела 5 — тяжелее и менее гибкие, чем Lottie. Использовать, только если 3D-путь окончательный. По умолчанию — Lottie (раздел 3).

6. ⚠ Важно: заставку (splash) нельзя анимировать

В приложении есть **native splash screen** — та картинка, что показывается **в первую секунду** запуска, пока грузится код. Настроена в `app.json`:

```
"splash": {
  "image": "./assets/images/splash-icon.png",
  "resizeMode": "contain",
  "backgroundColor": "#ffffff"
}
```

Это всегда статичное изображение (PNG). Ни iOS, ни Android не дают проигрывать там анимацию — таково ограничение платформ, это не лень разработчика.

Как это выглядит для пользователя: 1. **0–1 сек**: статичный splash (PNG-логотип). Не анимируется. 2. **1 сек и дальше**: приложение загрузилось → показываем **наш анимированный интро-экран** (Lottie из раздела 3.1) уже внутри приложения.

Что нужно от дизайнера, чтобы стык был плавным: - Первый кадр Lottie-интро должен **совпадать** с картинкой splash (та же поза маскота, тот же фон), тогда переход «статика → анимация» будет незаметным. - Фон splash (`backgroundColor`) и фон интро — **один и тот же цвет** (раздел 8).

7. Что должен сделать разработчик (не дизайнер) — ✅ СДЕЛАНО

Обязка уже в коде (коммит с mascot-анимацией). Ниже — что именно сделано, для истории. Замена заглушек на финал — раздел 7.5.

Инженерная задача, чтобы `.json` заработал:

1. Установить пакет: `npx expo install lottie-react-native` (совместим с текущим Expo SDK 54; ставит нативную часть, нужен пересбор dev-build).

2. Положить файлы в `Mobile/assets/animations/` (папку создать): - `Mobile/assets/animations/intro-mascot.json` - `Mobile/assets/animations/loader-mascot.json`
3. Интро-компонент — показать один раз при старте после native splash (`expo-splash-screen` → `hideAsync()` → Lottie-интро).
4. Лоадер — заменить текущие спиннеры на Lottie-луп там, где грузятся данные.

Технические анимации интерфейса (открытие ачивки, вращение винила в UI) **уже** сделаны на `react-native-reanimated` + `react-native-svg` — их не трогаем, речь только про маскот-ролики.

7.5. Как заменить заглушки на финальные анимации (пошагово)

Инженерная обвязка **уже стоит** (сделано, шаг 7). Сейчас в приложении крутятся **временные заглушки** — простое синее кольцо и диск в бренд-цвете. Когда аниматор пришлёт готовые `.json` — их надо просто **подложить вместо заглушек**. Код менять не нужно.

Что лежит сейчас:

```
Mobile/assets/animations/  
├─ intro-mascot.json    ← заглушка (в поле "nm": "...-PLACEHOLDER")  
├─ loader-mascot.json  ← заглушка  
└─ README.md
```

Шаги замены (делает разработчик, 5 минут):

1. Получить от аниматора два файла, проверить их на lottiefiles.com/preview — играют как надо, вес в пределах ТЗ (\$3).
2. **Перезаписать** файлы, **имена оставить те же**: - финальный интро → `Mobile/assets/animations/intro-mascot.json` - финальный лоадер → `Mobile/assets/animations/loader-mascot.json` - ⚠ Именно перезаписать, не удалять и не переименовывать — код грузит их по этим путям через `require()`. Другое имя = приложение не соберётся.
3. Проверить, что в новых файлах в поле `"nm"` **нет** суффикса `-PLACEHOLDER` (маркер заглушки). Если есть — прислали не тот файл.
4. Пересобрать приложение: - `.json` — это JS-ассет, нативного пересбора **не требует**. Хватает перезапуска Metro-бандлера (`npm expo start -c`, флаг `-c` чистит кэш). - Нативный пересборка нужна была **один раз** — при первой установке `lottie-react-native` (уже сделано). Дальше замена `.json` — без rebuild.
5. Прогнать глазами: холодный старт → интро; экран загрузки → лоадер.

Проверка «заглушка или финал» из терминала:

```
grep -l PLACEHOLDER Mobile/assets/animations/*.json  
# пусто = финальные анимации на месте  
# выводит путь = там всё ещё заглушка
```

Куда воткнуть лоадер в коде (если ещё не сделано на момент замены): - Компонент готов: `import { MascotLoader } from '@components/MascotLoader'`. - Ставить `<MascotLoader size={120} />` вместо

`ActivityIndicator` на экранах загрузки (коллекция, маркет, поиск). Он сам крутит Lottie, а в Expo Go падает на винил.

8. Фирменные цвета (палитра Вертушки)

Чтобы маскот-анимация совпала с приложением. Источник — `Mobile/constants/theme.ts`.

Фон под маскота (splash + интро) — почти белый с холодком: - `#FAFBFF` — основной фон экрана (используй его же для `backgroundColor` splash вместо `ffffff`, чтобы не было резкого стыка)

Бренд-градиент маскота (сине-розовый, как на рисунке): - `#3B4BF5` — royalBlue (насыщенный синий) - `#5B6AF5` — electricBlue - `#8B9CF7` — periwinkle - `#C5B8F2` — lavender - `#F0C4D8` — softPink (розовый)

Акцент (центр пластинки / «горячее»): - `#E85A2A` — ember (оранжевый), редкий акцент. Использовать точно, не заливать им всё.

Тёмный бренд (навигация, глубина): - `#0A0B3B` — deepNavy - `#0B1438` — brand.navy

Шрифты приложения (если в анимации нужен текст — перевести в кривые, но стиль такой): - Заголовки-hero: **Rubik Mono One** - Всё остальное: **Inter**

9. Чек-лист перед сдачей (для дизайнера)

Отдавать разработчику, только когда всё :

- ☐ Два файла: `intro-mascot.json` и `loader-mascot.json`
 - ☐ Оба открываются и корректно играют на lottiefiles.com/preview
 - ☐ Лоадер — луп без рывка на стыке (посмотреть 3–4 повтора подряд)
 - ☐ Интро: вес < 200 КБ, лоадер: вес < 100 КБ
 - ☐ Нет растровых картинок внутри (кроме согласованного halftone-решения)
 - ☐ Нет эффектов AE (Blur/Glow/Shadow), Merge Paths, Expressions, живого текста
 - ☐ FPS = 30, артборды квадратные (1080 и 512)
 - ☐ Первый кадр интро совпадает с картинкой splash
 - ☐ Цвета — из палитры раздела 8
 - ☐ Фон интро = `#FAFBFF`
-

10. Короткое ТЗ одним блоком (копипаст аниматору)

ФОРМАТ: Lottie .json, экспорт из After Effects плагином Bodymovin 5.7+

ФАЙЛ 1 – ИНТРО (играет 1 раз при старте):

30 fps, 2–4 сек, артборд 1080×1080, вес < 200 КБ, в конце – статичная поза
первый кадр = картинка splash (плавный стык)

ФАЙЛ 2 – ЛОАДЕР (крутится при загрузке):

30 fps, 1–2 сек, БЕСШОВНЫЙ ЛУП (первый кадр == последний),
артборд 512×512, вес < 100 КБ

ТОЛЬКО векторные shape-слои. Анимировать: позицию, поворот, масштаб, opacity.

ЗАПРЕТ: растровые png внутри, эффекты AE (Blur/Glow/Shadow/Gradient Ramp),
Merge Paths, Auto-Trace, Expressions (запечь в кейфреймы), живой текст
(перевести в кривые), 3D-слои, сложные маски/matte.

Halftone (точки на теле): упростить до плоского градиента ИЛИ отдать
отдельным неанимируемым слоем – согласовать с разработчиком.

Палитра: синий #3B4BF5 → #5B6AF5 → #8B9CF7 → розовый #F0C4D8;
акцент ember #E85A2A (точечно); фон #FAFBFF.

ПРОВЕРКА перед сдачей: файл играет на lottiefiles.com/preview так,
как задумано. Как там – так и в приложении.